

LABORATORY OBSERVATION

Student Name: D.vijaya sri

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5-3

1. TITLE

Express.js Student Server

2. AIM

To create a basic Express.js server with the routes `/`, `/students`, and `/about`, where the `/students` route returns a list of at least five students and the `/about` route displays information about the application.

3. OBJECTIVE

The objectives of this experiment are:

- To understand Express.js server setup.
 - To create and configure an Express.js application.
 - To implement routes using the HTTP GET method.
 - To return student data in JSON format.
 - To send an HTML response for the home and about routes.
 - To understand basic request and response handling.
-

4. THEORY

Express.js is a web application framework for Node.js used to build servers and web APIs. It provides a simple way to define routes and send responses to clients.

A route specifies a URL path and an HTTP method that the server handles. In this application, GET routes are created for `/`, `/students`, and `/about`.

The request (`req`) object represents the incoming client request, while the response (`res`) object is used to send data back to the client. The `res.json()` method sends JSON data, while `res.send()` sends an HTML or text response.

5. ALGORITHM / PROCEDURE

1. Import the Express.js module.
2. Create an Express application instance.
3. Define the server port as 3000.
4. Create an array containing at least five student records.
5. Define the GET `/` route to display the home page.

6. Define the GET /students route to return the student array as JSON.
 7. Define the GET /about route to display application information.
 8. Start the Express.js server using app.listen().
 9. Run the application using Node.js.
 10. Open the routes in a browser and verify the responses.
-

6. PROGRAM

app.js

```
const express = require("express");

const app = express();

const PORT = 3000;

// Sample student data
const students = [

  { id: 1, name: "Rahul", rollNo: "101" },

  { id: 2, name: "Priya", rollNo: "102" },

  { id: 3, name: "Arjun", rollNo: "103" },

  { id: 4, name: "Kiran", rollNo: "104" },

  { id: 5, name: "Anjali", rollNo: "105" }

];

// Home route
app.get("/", (req, res) => {

  res.send(`

    <h1>Student Server</h1>

    <p>Welcome to the Express.js Student Server.</p>

    <p>Available routes:</p>

    <ul>

      <li><a href="/students">View Students</a></li>

      <li><a href="/about">About Application</a></li>
```

package.json

```
{
  "name": "q3-expressjs-student-server",
  "version": "1.0.0",
  "description": "Basic Express.js server with student and about routes",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

7. CODE EXPLANATION

Code	Explanation
require("express")	Imports the Express.js framework.
express()	Creates an Express application instance.
const students = [...]	Stores the five student records used by the server.
app.get("/", ...)	Defines the home route and sends the main application page.
app.get("/students", ...)	Defines the students route and returns student data as JSON.
res.json(students)	Sends the student array as a JSON response.
app.get("/about", ...)	Defines the about route and sends application information.
app.listen()	Starts the Express.js server on port 3000.

8. TEST CASES AND EXECUTION DATA

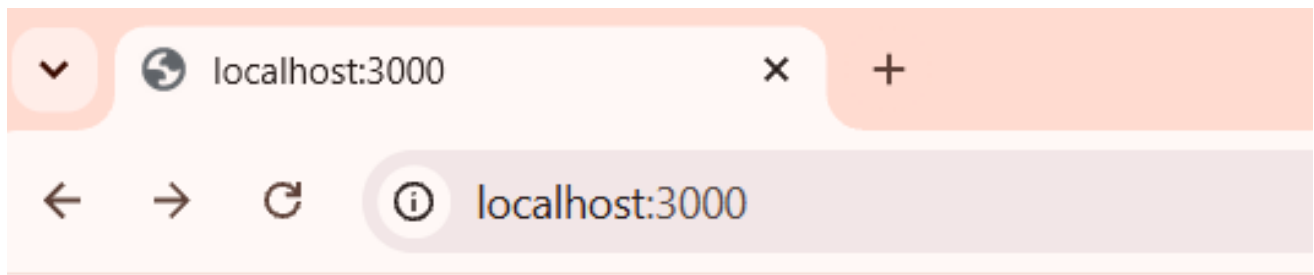
Test Case	Route	Expected Result	Actual Result	Status
TC-01	/	Home page displays Student Server and available routes.	Student Server page displayed.	Pass
TC-02	/students	A list of five students is returned in JSON format.	Five student records displayed.	Pass
TC-03	/about	Application information is displayed.	Screenshot not provided.	—
TC-04	Server startup	Server starts on port 3000 without errors.	Server running at localhost:3000.	Pass

9. OUTPUT

Output 1: Express.js Server Running in Terminal

```
c:\Users\dadhi\FULL_STACK\01-observation-and-record\observation\Task 5\Q3_ExpressJS_Student_Server\src>node app.js
Server running at http://localhost:3000
```

Output 2: Home Route (/)



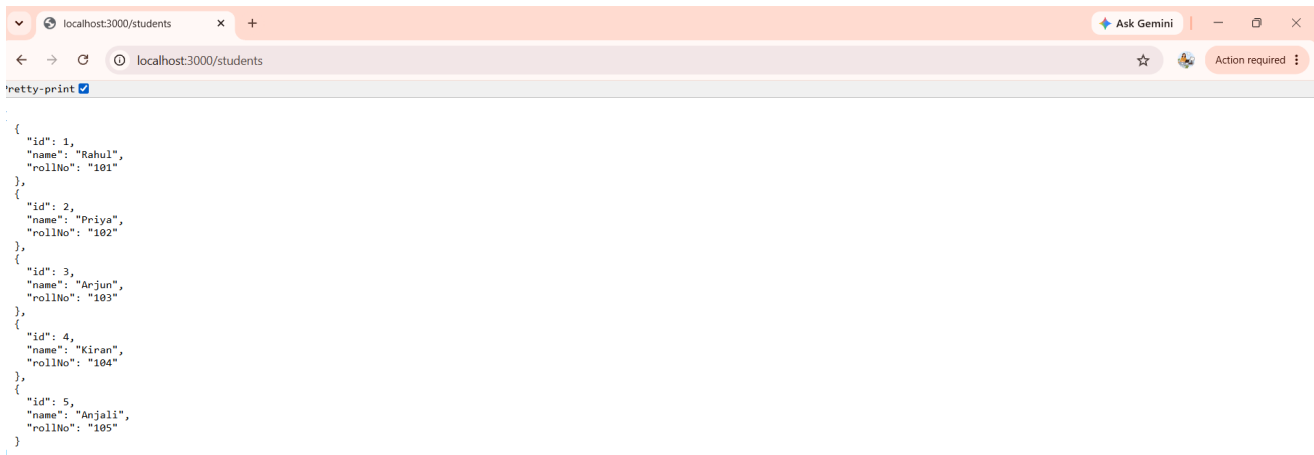
Student Server

Welcome to the Express.js Student Server.

Available routes:

- [View Students](#)
- [About Application](#)

Output 3: Students Route (/students)



Note: An /about output screenshot was not supplied, so no screenshot has been invented or claimed as executed in this document.

10. RESULT

Thus, the Express.js Student Server was implemented with the required /, /students, and /about routes. The supplied execution screenshots verify that the server starts successfully on port 3000, the home route displays correctly, and the /students route returns five student records in JSON format.

How this would be submitted

Source Code

```
Q3_ExpressJS_Student_Server/  
├── src/  
│   └── app.js  
├── package.json  
└── README.md
```